

FruitGame - Dokumentation

Stand: 8. Oktober 2026

Ursprungsidee

Ein Browserspiel im Stil von Fruit Ninja, gesteuert nicht mit Maus oder Touch, sondern mit echten Handbewegungen vor der Webcam. Bunte Objekte fliegen von unten ins Bild, man "schneidet" sie mit der Hand, eine Bombe darf nicht getroffen werden, am Ende zählt der Punktestand.

Der initiale Prompt

Der erste Auftrag kam als Stichpunktliste, sinngemäß:

- Objekte (erstmal bunte Kugeln) fliegen von unten nach oben
- Objekte können durch schnelle Handbewegungen zerstört (in zwei Hälften) werden
- Zwischendurch kommt eine Bombe, diese darf nicht zerstört werden
- Die Objekte geben jeweils einen Punkt, die Bombe gibt -3 Punkte
- Die Punkte werden gezählt und während der Runde oben links angezeigt
- Steuerung per Handbewegungen über Kamera
- Eine Runde geht 60 Sekunden
- Nach einer Runde Startscreen mit Top-10-Highscores ohne Namen, neue Runde startet durch Heben beider Hände
- Für die Gestenerkennung soll ein bereits fertiges Modell verwendet werden

Was daraus generiert wurde

Eine einzelne, in sich geschlossene HTML-Datei (`fruitgame.html`), die lokal im Browser läuft. Kein Build-Prozess, kein Server-Backend nötig außer einem simplen Webserver für die Dateiauslieferung. Die Gestenerkennung läuft über das fertige **MediaPipe Hand Landmarker**-Modell von Google, das zur Laufzeit von einem CDN geladen wird. Spielfeld und Objekte sind auf einem HTML5-Canvas gezeichnet.

Was im Verlauf dazukam

Wichtig zur Einordnung: Es gab in diesem Gesprächsverlauf keine gemeldeten Bugs und damit auch keine klassischen Fehlerkorrekturen, die Code getestet und als defekt gemeldet wurde. Das Spiel wurde ausschließlich auf JavaScript-Syntaxfehler geprüft (`node --check`), aber **nicht tatsächlich im Browser mit Kamera durchgespielt**. Die folgenden Punkte waren funktionale Erweiterungen auf expliziten Wunsch, keine Bugfixes:

- **Spielende nach 3 Bombentreffern:** drei Herzen im HUD, Runde endet sofort bei null Herzen, zusätzlich weiterhin reguläres Ende nach 60 Sekunden
- **Neue Sprites:** sechs komplett runde, auf dem Canvas prozedural gezeichnete Früchte (Orange, Apfel, Wassermelone, Kiwi, Limette, Pflaume) statt einfarbiger Kugeln, inklusive passender Schnittflächen für die beiden Hälften
- **Drei Schwierigkeitsgrade** (Leicht/Mittel/Schwer): unterschiedliches Tempo, unterschiedlicher Bombenanteil, unterschiedlicher Spawn-Abstand
- **Combo-System:** Zähler für Früchte in Folge ohne Bombentreffer und ohne durchfallende Frucht, mit Bonuspunkten alle 5 Treffer
- **Schwierigkeitsauswahl per Geste:** ursprünglich reichte Berühren des Buttons, auf Wunsch umgestellt auf Halten über dem Button für eine feste Dauer, analog zur Start-Geste

Offen und nur besprochen, aber noch nicht umgesetzt: eine Variante, bei der die Kamera klar sichtbar bleibt (keine Abdunklung) und die Objekte nur darüber eingeblendet werden.

Wie das Spiel aussieht und funktioniert

- **Startscreen:** zeigt die letzte Rundenbewertung (Grund: Zeit abgelaufen oder 3 Bomben getroffen), die Top-10-Liste der gewählten Schwierigkeit ohne Namen, sowie drei Schwierigkeits-Buttons
- **Rundenstart:** beide Handgelenke müssen oberhalb einer Linie im oberen Bildbereich erkannt werden und dort für eine feste Haltezeit bleiben, ein Fortschrittsbalken zeigt das Halten an
- **Schwierigkeitswahl:** gleiches Prinzip, Finger über dem gewünschten Button halten statt ihn nur zu berühren

- **Spielrunde:** Früchte und gelegentlich eine Bombe fliegen wie bei einem Wurf nach oben und fallen wieder, Treffer werden per Zeigefingerspitze registriert, wenn die Bewegung zwischen zwei Frames schnell genug durch das Objekt geht
- **HUD:** Punktestand und Countdown oben, darunter drei Herzen für verbleibende Bomben-Toleranz, Combo-Anzeige ab dem dritten Treffer in Folge
- **Rundenende:** automatisch bei abgelaufener Zeit oder beim dritten Bombentreffer, danach zurück zum Startscreen mit aktualisierter Bestenliste

Verwendete Technologien

- **HTML5 Canvas 2D** für Rendering von Kamerabild, Früchten, Spuren und Texten
- **Vanilla JavaScript (ES-Module)**, keine Frameworks
- **MediaPipe Tasks Vision – Hand Landmarker** ([@mediapipe/tasks-vision](https://github.com/google/mediapipe/tree/master/tasks/vision)), geladen über jsDelivr-CDN, inklusive WASM-Laufzeit; GPU-Delegate mit automatischem Fallback auf CPU
- **getUserMedia** für den Kamerazugriff
- **localStorage** , getrennt pro Schwierigkeitsgrad, für die Top-10-Listen; rein clientseitig, kein Backend, kein geräteübergreifender Sync

Funktionsweise einzelner Aspekte

- **Flugbahn:** jedes Objekt bekommt eine zufällige Zielhöhe und eine daraus berechnete Anfangsgeschwindigkeit, danach wirkt simulierte Schwerkraft wie bei einem senkrechten Wurf
- **Schnitterkennung:** aus zwei aufeinanderfolgenden Positionen der Zeigefingerspitze wird eine Strecke gebildet; liegt ein Objektmittelpunkt nah genug an dieser Strecke und war die Geschwindigkeit über dem Schwellwert, gilt es als geschnitten
- **Hand-Tracking zwischen Frames:** die Zeigefingerspitzen aus dem aktuellen und dem vorherigen Frame werden über die kürzeste Distanz einander zugeordnet, damit eine durchgehende Spur und eine Geschwindigkeit berechnet werden kann
- **Combo-Logik:** Zähler steigt bei jedem Fruchttreffer, wird bei Bombentreffer oder einer durchfallenden, ungeschnittenen Frucht auf 0 zurückgesetzt; alle 5 Treffer gibt es einen festen Bonus
- **Schwierigkeit:** ein globaler Geschwindigkeitsfaktor skaliert die Physik, ein zweiter Faktor den Abstand zwischen Spawns, ein dritter die Bombenwahrscheinlichkeit je Objekt

- **Highscores:** nach Rundenende wird der Score in die Liste der aktuellen Schwierigkeit einsortiert, auf 10 Einträge gekürzt und in `localStorage` unter einem schwierigkeitsspezifischen Schlüssel gespeichert

Was noch geändert werden könnte

- **Durchsichtiger Kamerahintergrund:** die Abdunklung des Kamerabilds entfernen oder stark reduzieren, damit die reale Umgebung sichtbar bleibt und nur Früchte, Bombe und HUD überlagert werden; dafür müsste Text- und Herzdarstellung kontraststabiler werden (z. B. stärkere Outlines)
- **Soundeffekte** für Schnitt, Bombentreffer, Rundenstart und -ende
- **Objektgröße abhängig von der Schwierigkeit**, nicht nur Tempo und Bombenanteil
- **Mehr visuelles Feedback** (Partikel, Bildschirmwackler bei Combo-Meilensteinen)
- **Pause-Funktion**, z. B. per Geste oder Taste
- **Persistente, geräteübergreifende Highscores** über ein kleines Backend auf dem vorhandenen Heimserver statt nur `localStorage`
- **Kalibrierung** der Handgröße/Distanz zu Spielbeginn für robustere Schnitterkennung bei unterschiedlichen Kameraabständen
- **Tatsächlicher Test im Browser mit echter Kamera**, inklusive Feintuning von Schwellwerten wie Schnittgeschwindigkeit und Spawn-Rate anhand realer Beobachtung

Mögliche Formulierung für einen Folge-Prompt, falls die nächsten Schritte angegangen werden sollen:

Mach den Kamerahintergrund durchsichtig (keine Abdunklung mehr), Sorge aber dafür, dass Punktestand, Herzen, Combo-Text und die Bestenliste auf hellem wie dunklem Hintergrund gut lesbar bleiben, z. B. durch kräftigere Outlines oder einen halbtransparenten Hintergrund nur hinter dem HUD.